

Planificación para el obligatorio

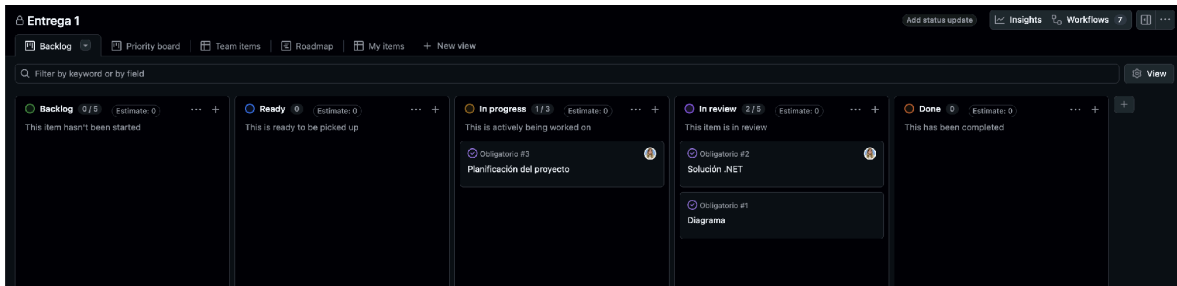
Etap a	Períod o	Actividad/Hito	Requerimientos/Entregable s	Responsabl e
1	02/09 – 10/09	Diseño del diagrama de clases y definición de la arquitectura de la solución	Diagrama con el diseño y arquitecturas elegidas	Cristian
		Creación y organización de la estructura inicial de las soluciones .NET según la arquitectura definida	Solución .NET	Ambos
		Definición de etapas, períodos, actividades, entregables y responsables del proyecto	Planificación del proyecto	Isabella
2	11/09 – 24/09	Implementación del login y logout de administradores en la aplicación de administración	RF01	Isabella
		Implementación del alta de usuarios y asignación de roles	RF02	Cristian
		Implementación del CRUD de historias y validaciones para su gestión y publicación	RF03	Ambos
3	25/09 -11/10	Implementación del login y logout mediante JWT	RF06	Isabella

		para la aplicación de lectura		
		Implementación de la lectura de historias, selección de opciones y registro del final alcanzado	RF07	Ambos
		Implementación del historial de lecturas del usuario con filtro por mes y año	RF08	Cristian
4	12/10 – 21/10	Implementación de la consulta del final más alcanzado de cada historia	RF10	Isabella
		Implementación del ranking de usuarios según historias distintas completadas	RF11	Ambos
		Investigación e implementación de los estilos visuales de lectura	RF12	Cristian
5	22/10 – 12/11	Integración, revisión y preparación de la solución, documentación, código y archivos para la entrega final	Entrega final en gestión	Ambos

Para organizar el trabajo y realizar el seguimiento del avance del proyecto se utilizará un tablero Kanban en GitHub Projects. Las actividades se organizarán según su estado (pendiente, pronta para comenzar, en progreso, en revisión y

finalizada), permitiendo visualizar el progreso y distribuir las tareas entre los integrantes del equipo.

Las tareas se irán agregando al tablero a medida que comienza cada etapa.



Ejemplo de tablero para Etapa 1.

Arquitecturas Elegidas

Para el desarrollo del sistema "Escoge tu propia aventura" se adoptó la Arquitectura Limpia (Clean Architecture) como base estructural de ambas soluciones. Esta arquitectura organiza el código en capas concéntricas con dependencias que apuntan siempre hacia el interior: en el núcleo se ubica la capa de Dominio, que contiene las entidades del sistema (Historia, Capítulo, Lectura, entre otras) con sus reglas de negocio y validaciones propias; por encima se encuentran los Casos de Uso (lógica de aplicación), y hacia el exterior las capas de Infraestructura y Presentación. Esta separación garantiza que el dominio sea completamente independiente de frameworks, bases de datos y detalles de implementación, facilitando el mantenimiento y la extensibilidad del sistema a lo largo del tiempo.

En cuanto al diseño del dominio, se aplicó Domain Driven Design (DDD), modelando las entidades centrales con jerarquías de herencia que reflejan el negocio: la clase abstracta Capítulo se especializa en CapítuloIntermedio y CapítuloFinal, mientras que Usuario se extiende en Administrador y Lector. Los estados de la historia se modelan igualmente mediante herencia, en lugar de enumeraciones, siguiendo el principio Open/Closed. Se implementa la interfaz IValidable en las entidades del dominio para que cada una sea capaz de validarse a sí misma, y se utilizan Value Objects donde corresponda. Las excepciones de negocio son propias del dominio y se propagan hacia las capas externas, que las traducen en mensajes descriptivos para el usuario o códigos HTTP según corresponda.

Los principios SOLID guían todas las decisiones de diseño a lo largo del proyecto. El principio de Responsabilidad Única se respeta separando claramente repositorios, servicios y controladores; el principio de Sustitución de Liskov

sustenta las jerarquías de herencia mencionadas; y el principio de Inversión de Dependencias se aplica mediante inyección de dependencias, de modo que las capas internas dependen de interfaces y no de implementaciones concretas. La solución se divide en dos proyectos independientes: la Aplicación de Administración (ASP.NET MVC con autenticación propia, sin Web API expuesta) y la Aplicación de Lectura (Web API RESTful con autenticación JWT + cliente MVC que consume la API mediante HttpClient). Ambas soluciones comparten la misma filosofía arquitectónica y utilizan Entity Framework 10 con LINQ en sintaxis de método para el acceso a datos.